CHAPTER-6
COMBINATIONAL LOGIC CIRCUITS

6.1 Introduction

Logic circuits can be classified into two major categories: Combinational logic circuits and Sequential logic circuits.

A combinational logic circuit is defined as a combination of logic devices whose output is a function of the present values of the input variables and independent of the previous values. That is, in short, the outputs of a combinational logic circuit at any instant of time are dependent only on the inputs present at that instant of time. These circuits use logic gates like AND, OR, and NOT gates and don't have a memory characteristics.

Fig.1 shows the block diagram of a combinational logic circuit. The circuit has some inputs, logic gates inside the box for processing inputs, and some outputs.

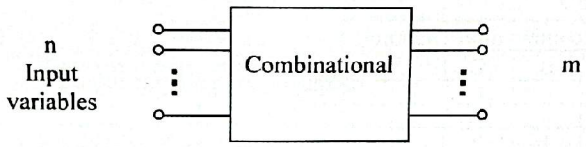


Fig.1: Block diagram of a combinational logic circuit

A combinational logic circuit accepts inputs from some external sources, then process the inputted data, and finally, the required output data will be available on the data output lines.

The operation performed by a combinational logic circuit can be specified logically by a set of m output functions, where each function is of n input variables. For n input variables there are 2^n possible input combinations and for each combination there is one possible output value. Therefore, a combinational logic circuit can also be specified with a truth table that lists the output values for each input combination.

6.2 Designing Combinational Logic Circuits

The design procedure of combinational logic circuits is usually a multi-step process. It starts from the specification of the problem and ends through the realization of the logic circuit. The following steps are involved in designing combinational logic circuits:

CamScanner

1. Analyze the problem intuitively.
2. Determine the number of inputs and outputs and assign a letter symbol to each.
3. Develop the truth table that shows the relationship between inputs and outputs.
4. Write the output expression in sum-of-products form.
5. Simplify the output expression and
6. Implement the logic circuit for the simplified expression.

The following examples illustrate the complete design procedure.

Example 1. Design a combinational logic circuit with three inputs A, B, and C and an output that goes LOW only when A is HIGH while B and C are different.

Solution: Three input signals: A, B, and C; One output signal: Y

Output: Y=LOW ; if A=HIGH and B≠C.
=HIGH ; otherwise.

We now develop the truth table below:

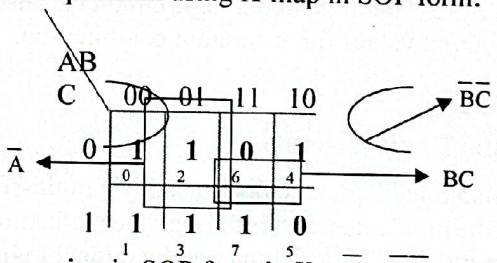
Truth table

Input combination			Output, Y
A	B	C	
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Now write the output expression in SOP form. Consider the input combinations (minterms) for which Y=1. In this case, Y will be in Sum-of-Products form as follows:

$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + \overline{A}BC + A\overline{B}C + ABC$$

Simplify the output expression using K-map in SOP form:



The simplified expression in SOP form is $Y = \overline{A} + \overline{B}\overline{C} + BC$.

Now implement the simplified expression:

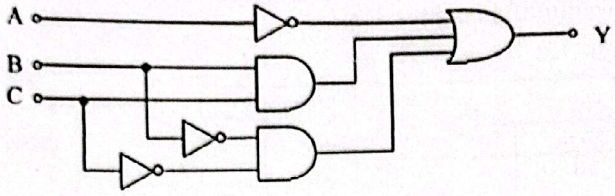


Fig.2: Implementation of the logic circuit for the simplified expression

Example 2. Design a combinational logic circuit that controls the passage of a signal A according to the following requirement:

i. Output Y will equal A when control inputs B and C are same.
ii. Y will remain HIGH when B and C are different.

Solution: Three input signals: A, B, and C; One output signal: Y

Output: Y=A ; if B=C.
=HIGH ; if B≠C.

Now develop the truth table below:

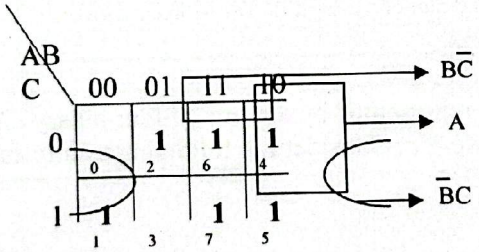
Truth table

Input combination			Output, Y
A	B	C	
0	0	0	0 (=A)
0	0	1	1
0	1	0	1
0	1	1	0 (=A)
1	0	0	1 (=A)
1	0	1	1
1	1	0	1
1	1	1	1 (=A)

Now write the output expression in SOP form. Consider the input combinations (minterms) for which Y=1. In this case, Y will be in Sum-of-Products form as follows:

$$Y = \overline{A}BC + A\overline{B}\overline{C} + AB\overline{C} + A\overline{B}C + ABC$$

Simplify the output expression using K-map in SOP form:



The simplified expression in SOP form is $Y = A + \overline{B}C + B\overline{C}$.

Now implement the simplified expression:

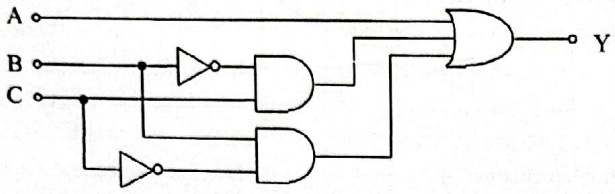


Fig.3: Implementation of the logic circuit for the simplified expression

6.3 Code Converters

There are many codes used in digital systems. SBC, BCD, Gray, Excess-3, Octal, ASCII, and EBCDIC are common examples of these codes. Some of these codes are numeric and some are alphanumeric. Therefore, for processing it may often require to convert one code into another. The logic circuits that are used for this purpose are known as code converters. For example, BCD-to-Binary and Binary-to-BCD converter, Binary-to-Gray and Gray-to-Binary converter, ASCII-to-EBCDIC and EBCDIC-to-ASCII converter etc. Now we will discuss some of these code converters.

6.3.1 BCD-to-Binary Converter

A BCD-to-Binary converter is a logic circuit that converts a BCD number to binary. For example, if the BCD input is $00100111)_{BCD}$, the converter will produce $11011)_2$ (See Fig.4).

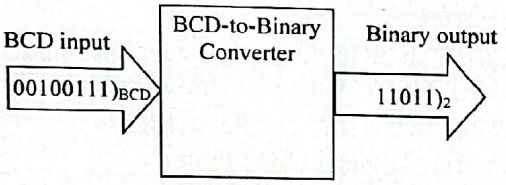
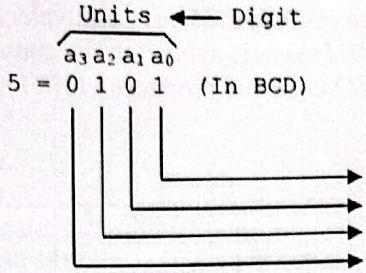


Fig.4: Block diagram of a 2-digit BCD-to-Binary converter

Conversion process

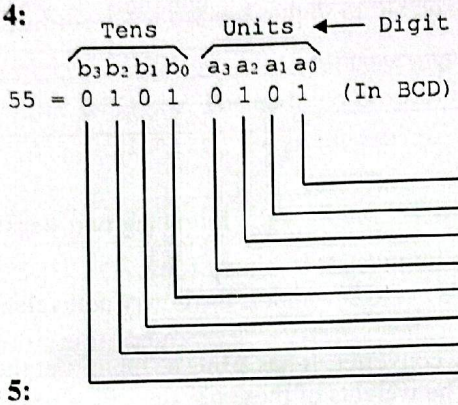
In BCD, each decimal digit is represented by a group of four binary digits. So, each bit in BCD must have a specific weight. Consider the following examples for determining the weights of each bit in BCD.

Example 3:



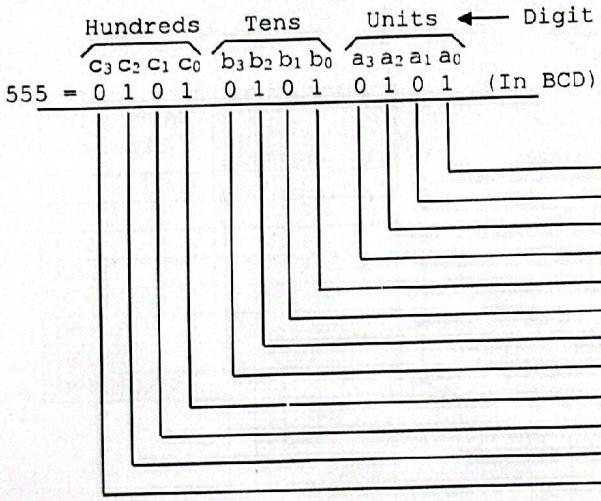
Weight of each bit in Decimal	Weight in Binary
1	1
2	10
4	100
8	1000

Example 4:



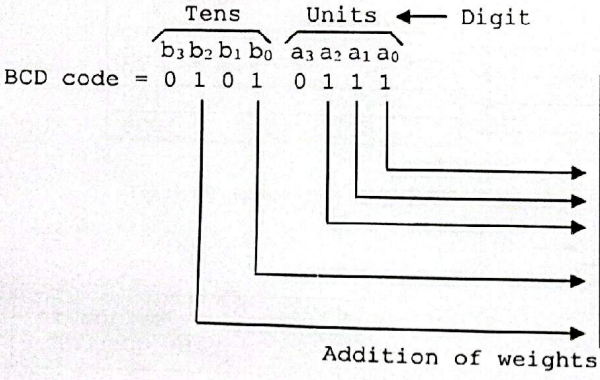
Weight of each bit in Decimal	Weight in Binary
1	1
2	10
4	100
8	1000
10	1010
20	10100
40	101000
80	1010000

Example 5:



Weight of each bit in Decimal	Weight in Binary
1	1
2	10
4	100
8	1000
10	1010
20	10100
40	101000
80	1010000
100	1100100
200	11001000
400	110010000
800	1100100000

From the above examples, we see that each bit in BCD has a specific weight. If the weights of all the 1's in BCD are added, the result is the binary equivalent to the BCD code. For example, if the BCD input $01010111)_{BCD}$ is applied to the converter, it only adds the binary weights of all 1's in the BCD code and produces $111001)_2$.

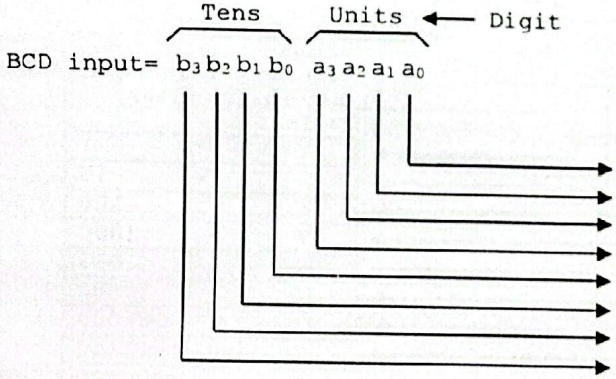


Weight of each 1 (in Binary)	Weight (in Decimal)
000001	1
000010	2
000100	4
001010	10
101000	40
	57

So, it is now clear that the conversion process involves the following two steps:

1. Represent the weight of each bit in BCD in binary form.
2. Add the weights for all the 1's in BCD which is the binary equivalent to the BCD number.

Let us consider a 2-digit BCD-to-Binary converter. It has 8 bits as input. Let these bits be defined by $b_3, b_2, b_1, b_0, a_3, a_2, a_1, a_0$. The weights of these bits are given below.



Weight of each bit in Decimal	Weight in Binary
1	1
2	10
4	100
8	1000
10	1010
20	10100
40	101000
80	1010000

Rewriting the weights as follows:

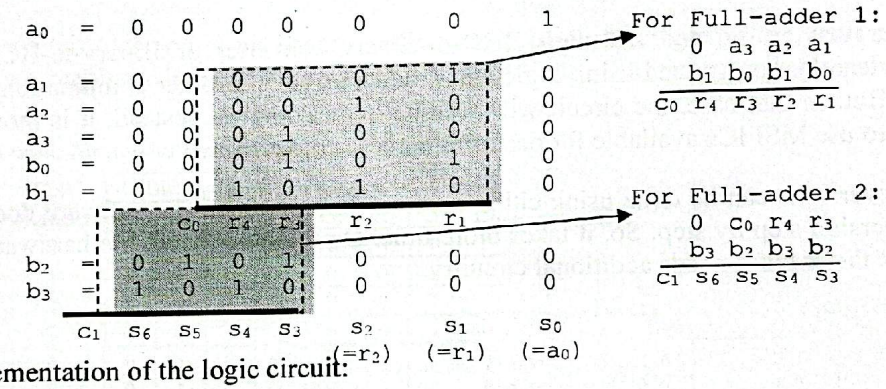
15/07/2026

CamScanner

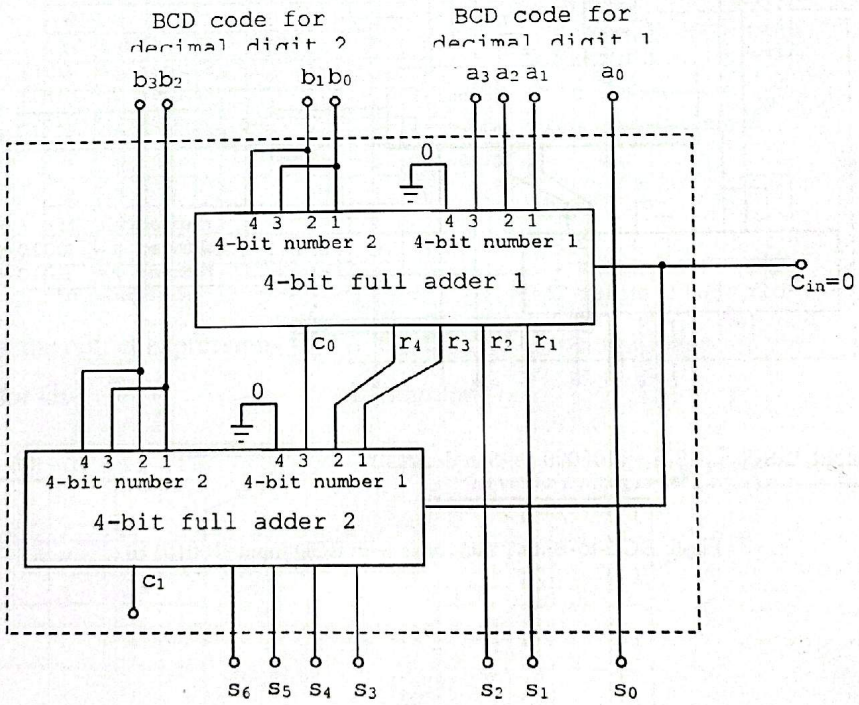
a_0	=	0	0	0	0	0	0	1	(=1 in decimal)
a_1	=	0	0	0	0	0	1	0	(=2 in decimal)
a_2	=	0	0	0	0	1	0	0	(=4 in decimal)
a_3	=	0	0	0	1	0	0	0	(=8 in decimal)
b_0	=	0	0	0	1	0	1	0	(=10 in decimal)
b_1	=	0	0	1	0	1	0	0	(=20 in decimal)
b_2	=	0	1	0	1	0	0	0	(=40 in decimal)
b_3	=	1	0	1	0	0	0	0	(=80 in decimal)
		C_1	S_6	S_5	S_4	S_3	S_2	S_1	S_0 (Addition of weights)

Here the weight of each BCD bit has been represented with 7 bits, because to represent the largest weight (i.e. weight of b_3) at least 7 bits are needed. Now add these weights and $S_6S_5S_4S_3S_2S_1S_0$ is the desired output of the converter.

For a 2-digit BCD-to-Binary converter, at least two 4-bit full adders are required to add the weights of the BCD bits. The addition process is shown below.



Implementation of the logic circuit:



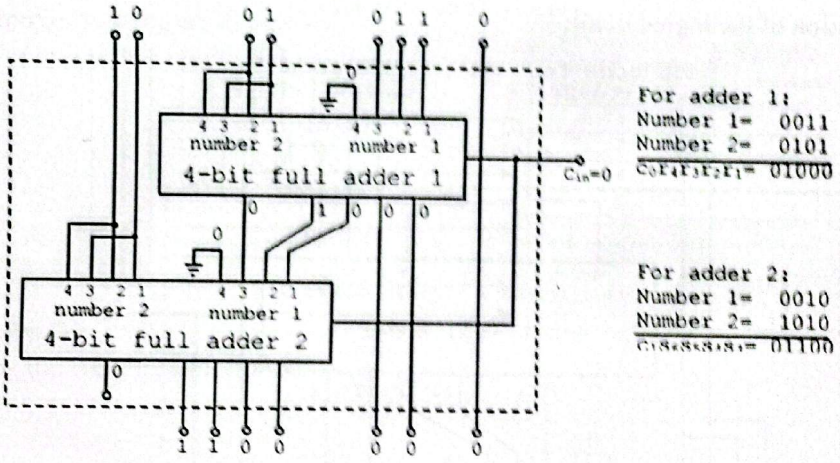
Binary equivalent output = $S_6S_5S_4S_3S_2S_1S_0$

Fig.5: Logic circuit of a 2-digit BCD-to-Binary converter

Suppose that a 2-digit BCD number 10010110 (=96 in decimal) is applied to the converter. Here $b_3b_2b_1b_0$ $a_3a_2a_1a_0$=1001 0110. Fig.6 shows this arrangement. The logic circuit produces binary output $S_6S_5S_4S_3S_2S_1S_0$=1100000 which is equivalent to the BCD input 10010110.

In similar way, a 3-, and a 4-digit BCD-to-Binary converter or Binary-to-BCD converter can be constructed using logic gates, adder circuits or other combinational circuits. But, in that case, the circuit will become more complex. Instead, it is more efficient to use MSI ICs available for performing these conversions.

Code conversions can be done using either hardware or software. The software does this conversion step by step. So, it takes more time. On the other hand, the hardware method is faster but it needs additional circuitry.

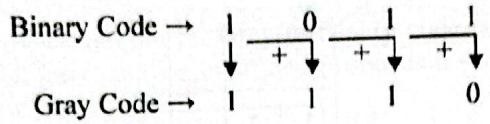


Binary output, $S_6S_5S_4S_3S_2S_1S_0$=1100000 (=96 in decimal)
which is equivalent to the BCD input 10010110

Fig.6: BCD-to-Binary converter with BCD input=10010110

6.3.2 Binary-to-Gray Code Converter

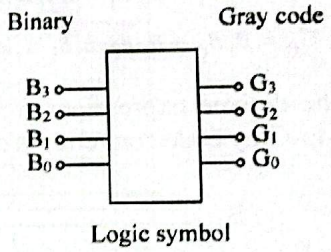
A Binary-to-Gray code converter is a logic circuit that takes input data represented in straight binary code and converts it to gray code. For example, if a binary number 1011 is applied to the converter, it will generate the gray code 1110. The conversion process is shown below.



Now we will design a 4-bit Binary-to-Gray code converter logic circuit. This circuit takes a 4-bit binary input and produces a 4-bit gray code. The truth table is given below.

Truth table for a 4-bit Binary-to-Gray converter

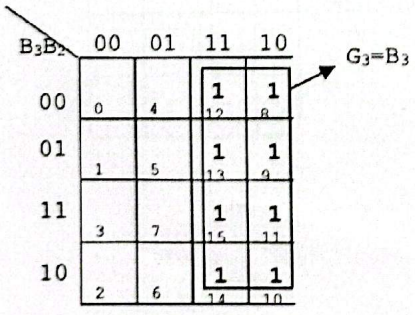
Cell Number	Binary Input				Gray Code Output			
	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0



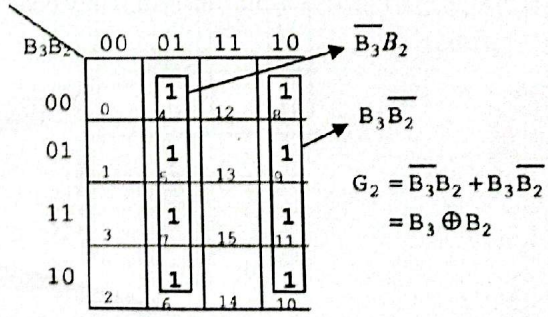
Logic symbol

Now derive the output expressions for G_3, G_2, G_1, and G_0 using K-map:

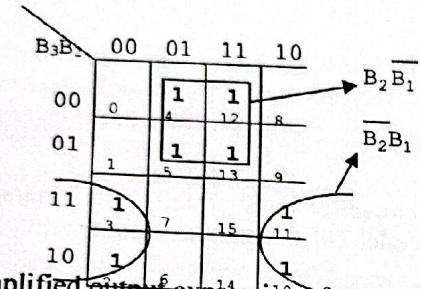
K-map for G_3



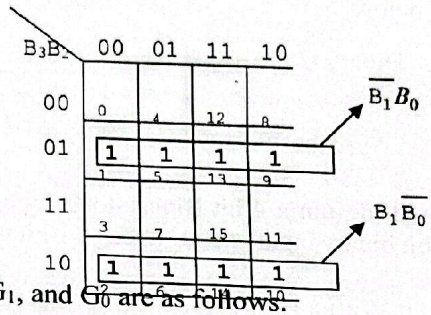
K-map for G_2



K-map for G_1



K-map for G_0



Simplified output expressions for G_3, G_2, G_1, and G_0 are as follows.

$$G_3 = B_3$$

$$G_2 = \overline{B_3}B_2 + B_3\overline{B_2} = B_3 \oplus B_2$$

$$G_1 = \overline{B_2}B_1 + B_2\overline{B_1} = B_2 \oplus B_1$$

$$G_0 = \overline{B_1}B_0 + B_1\overline{B_0} = B_1 \oplus B_0$$

From the derived expressions for G_3, G_2, G_1, and G_0, the logic circuit of the 4-bit Binary-to-Gray code converter is drawn in Fig.7 below.

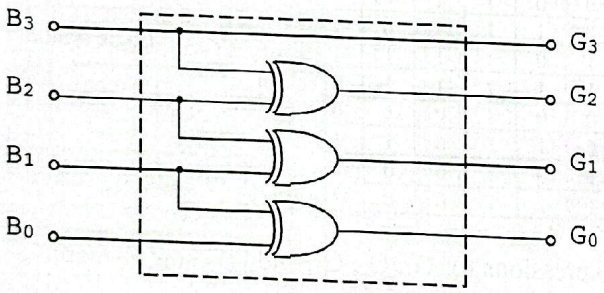
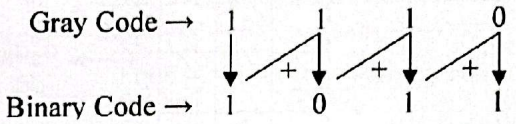


Fig.7: A 4-bit Binary-to-Gray code converter

6.3.3 Gray-to-Binary Code Converter

A Gray-to-Binary code converter is a logic circuit that accepts input data represented in gray code and converts it to binary code. For example, for a 4-bit gray code input of 1110, the converter will generate the binary code 1011. The conversion process is shown below.



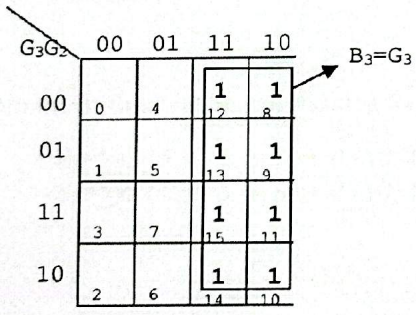
Now we will design a 4-bit Gray-to-Binary code converter logic circuit. This circuit takes a 4-bit gray code as input and produces a 4-bit binary code. The truth table is given below.

Truth table of a 4-bit Gray-to-Binary converter

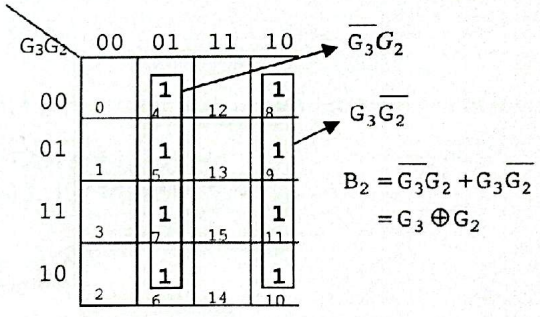
Cell Number	Gray code Input				Binary output			
	G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
3	0	0	1	1	0	0	1	0
2	0	0	1	0	0	0	1	1
6	0	1	1	0	0	1	0	0
7	0	1	1	1	0	1	0	1
5	0	1	0	1	0	1	1	0
4	0	1	0	0	0	1	1	1
12	1	1	0	0	1	0	0	0
13	1	1	0	1	1	0	0	1
15	1	1	1	1	1	0	1	0
14	1	1	1	0	1	0	1	1
10	1	0	1	0	1	1	0	0
11	1	0	1	1	1	1	0	1
9	1	0	0	1	1	1	1	0
8	1	0	0	0	1	1	1	1

Now derive the output expressions for B_3, B_2, B_1, and B_0 using K-map:

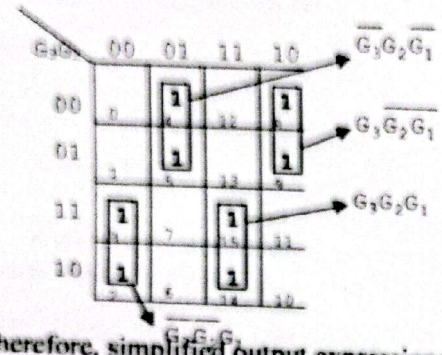
K-map for B_3



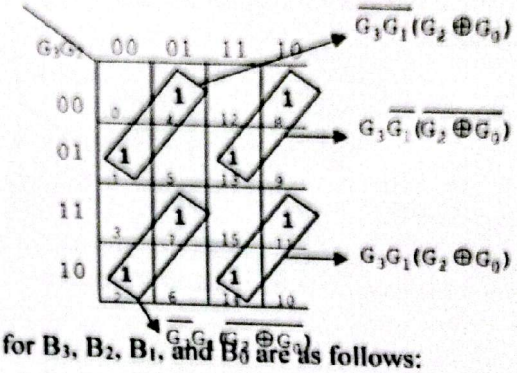
K-map for B_2



K-map for B_3



K-map for B_0



Therefore, simplified output expressions for B_3, B_2, B_1, and B_0 are as follows:

$$B_3 = G_3$$

$$B_2 = \bar{G}_3G_2 + G_3\bar{G}_2 = G_3 \oplus G_2$$

$$B_1 = \bar{G}_3G_2\bar{G}_1 + G_3\bar{G}_2\bar{G}_1 + G_3G_2G_1 + \bar{G}_3\bar{G}_2G_1$$
$$= \bar{G}_1(\bar{G}_3G_2 + G_3\bar{G}_2) + G_1(G_3G_2 + \bar{G}_3\bar{G}_2)$$
$$= \bar{G}_1(G_3 \oplus G_2) + G_1\overline{(G_3 \oplus G_2)}$$
$$= G_3 \oplus G_2 \oplus G_1$$

$$B_0 = \bar{G}_3\bar{G}_1(G_2 \oplus G_0) + G_3\bar{G}_1\overline{(G_2 \oplus G_0)} + G_3G_1(G_2 \oplus G_0) + \bar{G}_3G_1\overline{(G_2 \oplus G_0)}$$
$$= (G_2 \oplus G_0)(\bar{G}_3\bar{G}_1 + G_3G_1) + \overline{(G_2 \oplus G_0)}(G_3\bar{G}_1 + \bar{G}_3G_1)$$
$$= (G_2 \oplus G_0)\overline{(G_3 \oplus G_1)} + \overline{(G_2 \oplus G_0)}(G_3 \oplus G_1)$$
$$= (G_2 \oplus G_0) \oplus (G_3 \oplus G_1)$$
$$= G_3 \oplus G_2 \oplus G_1 \oplus G_0$$

From the derived expressions for B_3, B_2, B_1, and B_0, the logic circuit of the 4-bit Gray-to-Binary code converter is drawn in Fig.8 below.

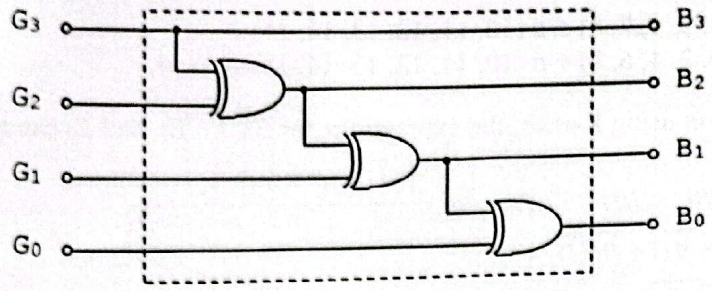


Fig.8: A 4-bit Gray-to-Binary code converter

6.3.4 BCD-to-Excess-3 Code Converter

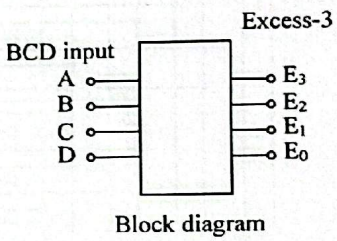
A BCD-to-Excess-3 code converter is a logic circuit that accepts data represented in BCD code and converts it to Excess-3 code. For example, if the BCD input code 0110 is applied, the converter will generate 1001. Fig.9 shows the truth table and the block diagram of a BCD-to-Excess-3 code converter.

Truth table of BCD-to-Excess-3 code conversion

BCD Input				Excess-3 output			
A	B	C	D	E_3	E_2	E_1	E_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	×	×	×	×
1	0	1	1	×	×	×	×
1	1	0	0	×	×	×	×
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×

Invalid BCD code

Don't care Condition



Block diagram

Fig.9

From the truth table the expressions for E_3, E_2, E_1, and E_0 can be written as given below:

$E_3=\sum m\ (5, 6, 7, 8, 9) + d\ (10, 11, 12, 13, 14, 15)$
$E_2=\sum m\ (1, 2, 3, 4, 9) + d\ (10, 11, 12, 13, 14, 15)$

$E_1 = \sum m\,(0, 3, 4, 7, 8) + d\,(10, 11, 12, 13, 14, 15)$
$E_0 = \sum m\,(0, 2, 4, 6, 8) + d\,(10, 11, 12, 13, 14, 15)$

After simplification using K-map, the expressions for E_3, E_2, E_1, and E_0 can be written as follows:

$$E_3 = A + BC + BD$$
$$E_2 = \overline{B}C + \overline{B}D + B\overline{C}\,\overline{D}$$
$$E_1 = CD + \overline{C}\,\overline{D}$$
$$E_0 = \overline{D}$$

From the derived expressions for E_3, E_2, E_1, and E_0, the logic circuit of the BCD-to-Excess-3 code converter is drawn in Fig.10.

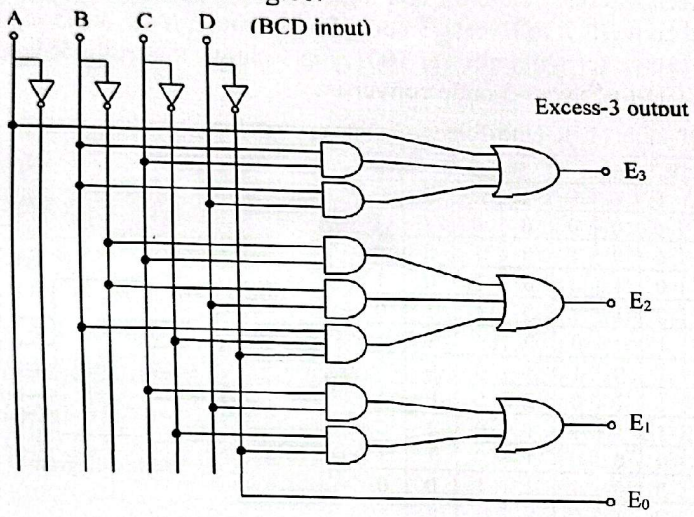


Fig.10: Logic circuit of BCD-to-Excess-3 code converter

The BCD-to-Excess-3 code converter can also be implemented using a 4-bit adder circuit. If we look at the truth table, we will see that the difference between the BCD code and the Excess-3 code is 3 (i.e. in binary 0011). In other words, Excess-3 code can be obtained by adding 3 to each BCD code. Fig.11 shows this implementation using a 4-bit adder circuit.

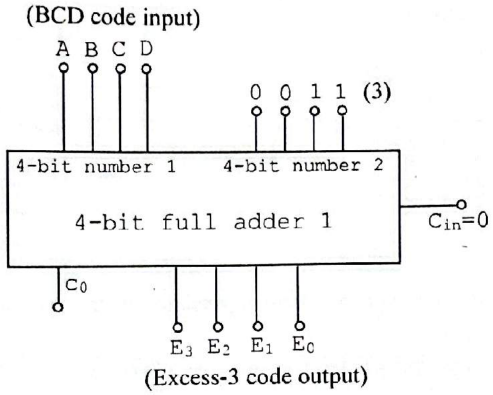


Fig.11: BCD-to-Excess-3 code converter using a 4-bit adder.

6.4 Digital Comparators

A digital comparator is a logic circuit that compares two binary numbers and generates an output that indicates whether the numbers are equal or which one is greater or less. Fig.12 shows the logic diagram of an n-bit comparator.

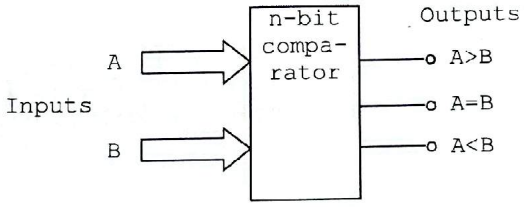


Fig.12: Logic diagram of an n-bit comparator

The comparator receives two n-bit numbers, A and B as inputs. Depending on the magnitudes of the numbers, only one of the three outputs will be HIGH.

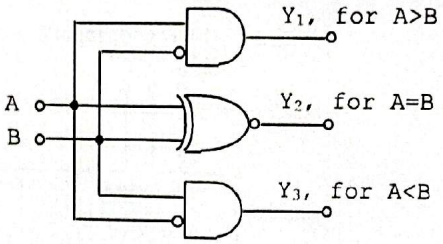
6.4.1 1-bit Comparator

A 1-bit comparator accepts two 1-bit numbers, A and B as inputs and produces an output indicating whether A>B, or A=B, or A<B. The truth table for a 1-bit comparator is given below.

Truth table: 1-bit comparator

Inputs		Outputs		
A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

(a)



(b)

Fig.13: (a) Truth table of a 1-bit comparator (b) Its logic circuit

From the truth table, the expressions for the outputs A>B, A=B, and A<B can be written as follows:

For A>B, $Y_1 = A\overline{B}$

For A=B, $Y_2 = \overline{A\overline{B}} + AB = \overline{A \oplus B} = A \odot B$

For A<B, $Y_3 = \overline{A}B$

The logic circuit of the 1-bit comparator is shown in fig.13. If the inputs AB=00 or AB=11 are applied to this circuit, it will produce Y_1=0, Y_2=1, and Y_3=0 indicating that the numbers are equal. Similarly, for AB=10 and AB=01, it will produce only Y_1=1 and Y_3=1 respectively.

6.4.2 2-bit Comparator

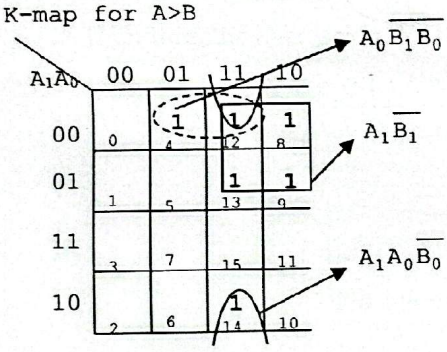
A 2-bit comparator receives two 2-bit numbers, A and B as inputs and produces an output indicating whether A>B, or A=B, or A<B. The truth table for a 2-bit comparator is given below.

Truth table of a 2-bit comparator

Inputs				Outputs		
Number A		Number B				
A_1	A_0	B_1	B_0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

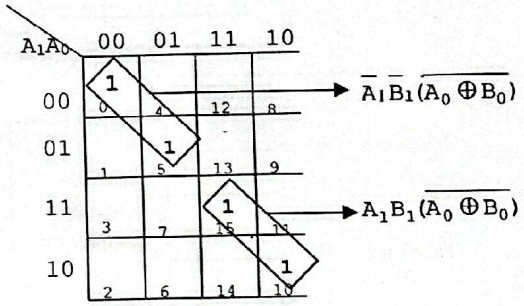
Now derive the output expressions for A>B, A=B, and A<B using K-map:

K-map for A>B



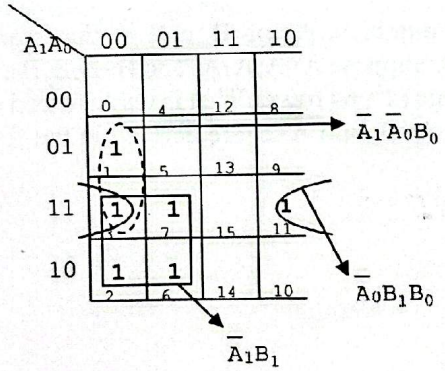
$$Y_1 = A_1\bar{B}1 + A_1A_0\bar{B}0 + A_0\bar{B}1\bar{B}0$$

K-map for A=B



$$Y_2 = A_1B_1(\overline{A_0 \oplus B_0}) + \bar{A}_1\bar{B}1(\overline{A_0 \oplus B_0})$$
$$= (\overline{A_0 \oplus B_0})\ (\overline{A_1 \oplus B_1})$$

K-map for A<B



$$Y_3 = \bar{A}_1B_1 + \bar{A}_0B_1B_0 + \bar{A}_1\bar{A}_0B_0$$

Now draw the logic circuit from the derived simplified output expressions:

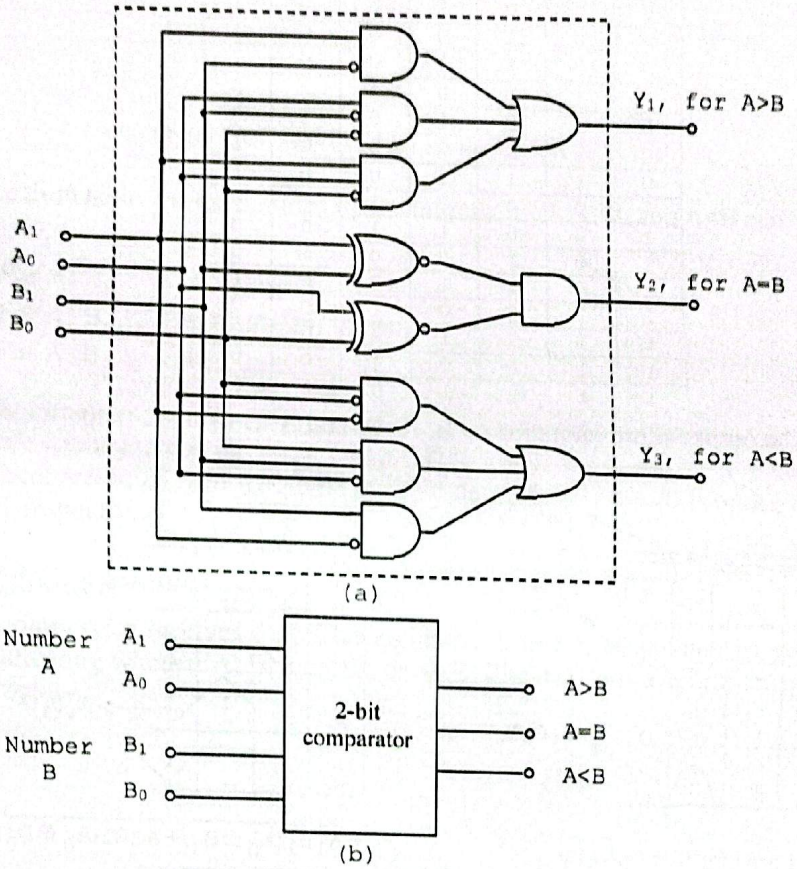


Fig.14: (a) logic circuit (b) logic symbol

6.4.3 3-bit Comparator

A 3-bit comparator compares two 3-bit numbers, A and B, and produces an output indicating whether A>B, or A=B, or A<B. Suppose $A=A_2A_1A_0$ and $B=B_2B_1B_0$. For this 3-bit comparator the truth table will be large (2^6=64 rows). That is why, instead of using the truth table, the expressions for A>B, A=B, and A<B are derived in the following way.

1. Expression for A>B:

The number $A_2A_1A_0$ will be greater than the number $B_2B_1B_0$,

if $A_2>B_2$
Or if $A_2=B_2$ but $A_1>B_1$
Or if $A_2=B_2$ and $A_1=B_1$ but $A_0>B_0$.

Therefore, the expression, Y_1 for A>B can be written as

$$Y_1 = A_2\overline{B}_2 + (A_2 \Theta B_2)A_1\overline{B}_1 + (A_2 \Theta B_2)(A_1 \Theta B_1)A_0\overline{B}_0$$

2. Expression for A=B:
The number $A_2A_1A_0$ will be equal to the number $B_2B_1B_0$,

if $A_2=B_2$, $A_1=B_1$ and $A_0=B_0$.

Therefore, the expression, Y_2 for A=B can be written as

$$Y_2 = (A_2 \Theta B_2)(A_1 \Theta B_1)(A_0 \Theta B_0)$$

3. Expression for A<B:
The number $A_2A_1A_0$ will be smaller than the number $B_2B_1B_0$,

if $A_2<B_2$
Or if $A_2=B_2$ but $A_1<B_1$
Or if $A_2=B_2$ and $A_1=B_1$ but $A_0<B_0$

Therefore, the expression, Y_3 for A>B can be written as

$$Y_3 = \overline{A}_2B_2 + (A_2 \Theta B_2)\overline{A}_1B_1 + (A_2 \Theta B_2)(A_1 \Theta B_1)\overline{A}_0B_0$$

Now draw the logic circuit from the derived expressions:

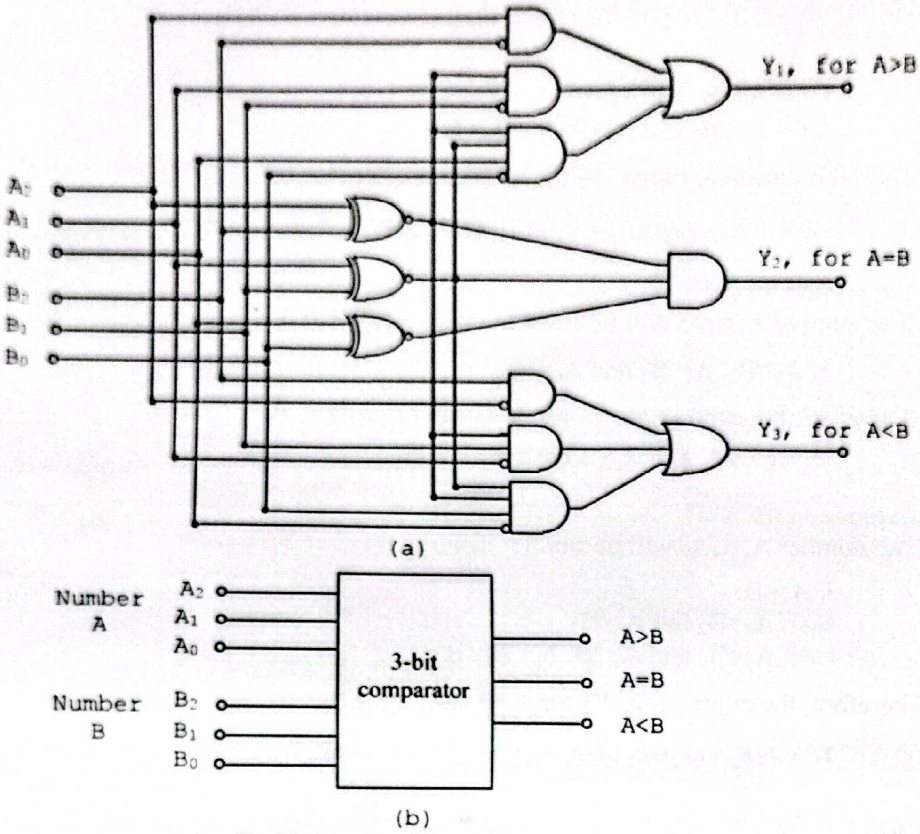


Fig.15: (a) logic circuit (b) logic symbol

6.4.4 4-bit Comparator

A 4-bit comparator compares two 4-bit numbers, A and B, and produces an output indicating whether A>B, or A=B, or A<B. Suppose $A=A_3A_2A_1A_0$ and $B=B_3B_2B_1B_0$. For this 4-bit comparator the truth table will be very large (2^8=256 rows). That is why, instead of using the truth table, the expressions for A>B, A=B, and A<B are derived in the following way.

1. Expression for A>B:
   The number $A_3A_2A_1A_0$ will be greater than the number $B_3B_2B_1B_0$,
   - if $A_3>B_3$
   - Or if $A_3=B_3$ but $A_2>B_2$
   - Or if $A_3=B_3$ and $A_2=B_2$ but $A_1>B_1$
   - Or if $A_3=B_3$, $A_2=B_2$, and $A_1=B_1$, but $A_0>B_0$.

   Therefore, the expression, Y_1 for A>B can be written as

$$Y_1 = A_3\overline{B}_3 + (A_3 \Theta B_3)A_2\overline{B}_2 + (A_3 \Theta B_3)(A_2 \Theta B_2)A_1\overline{B}_1 + (A_3 \Theta B_3)(A_2 \Theta B_2)(A_1 \Theta B_1)A_0\overline{B}_0$$

2. Expression for A=B:
   The number $A_3A_2A_1A_0$ will be equal to the number $B_3B_2B_1B_0$,
   if $A_3=B_3$, $A_2=B_2$, $A_1=B_1$ and $A_0=B_0$.

   Therefore, the expression, Y_2 for A=B can be written as
   $$Y_2 = (A_3 \Theta B_3)(A_2 \Theta B_2)(A_1 \Theta B_1)(A_0 \Theta B_0)$$

3. Expression for A<B:
   The number $A_3A_2A_1A_0$ will be smaller than the number $B_3B_2B_1B_0$,
   if $A_3<B_3$
   Or if $A_3=B_3$ but $A_2<B_2$
   Or if $A_3=B_3$ and $A_2=B_2$ but $A_1<B_1$
   Or if $A_3=B_3$, $A_2=B_2$, and $A_1=B_1$ but $A_0<B_0$

   Therefore, the expression, Y_3 for A>B can be written as
   $$Y_3 = \overline{A}_3B_3 + (A_3 \Theta B_3)\overline{A}_2B_2 + (A_3 \Theta B_3)(A_2 \Theta B_2)\overline{A}_1B_1 + (A_3 \Theta B_3)(A_2 \Theta B_2)(A_1 \Theta B_1)\overline{A}_0B_0$$

Now draw the logic circuit using the derived expressions

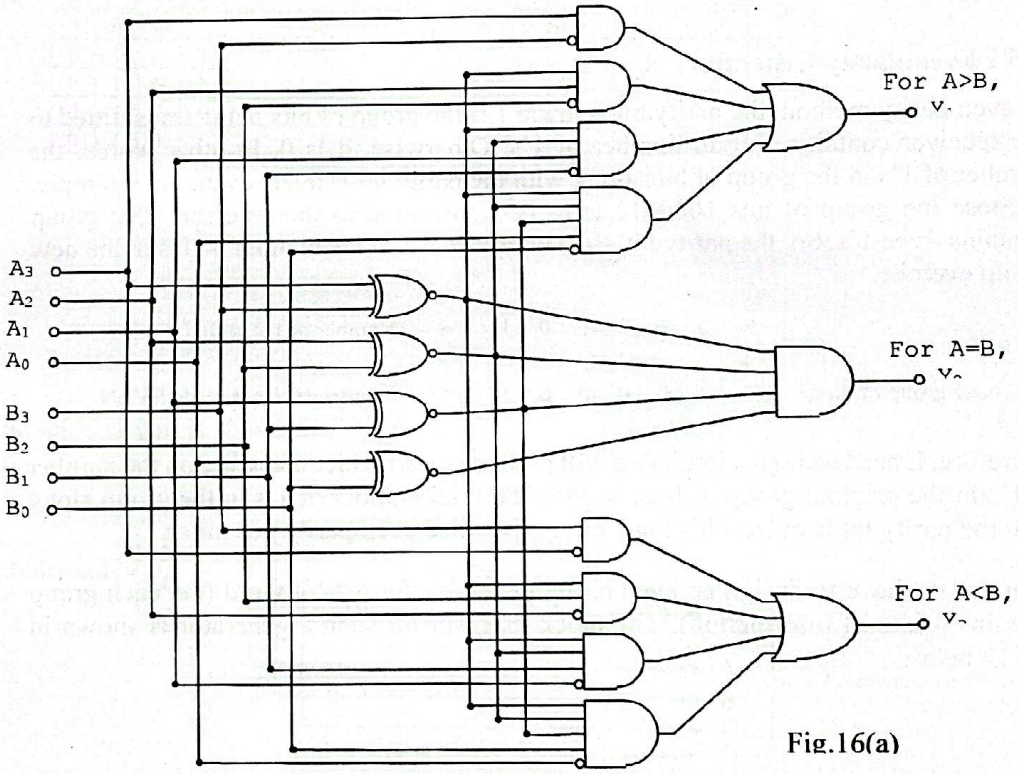


Fig.16(a)

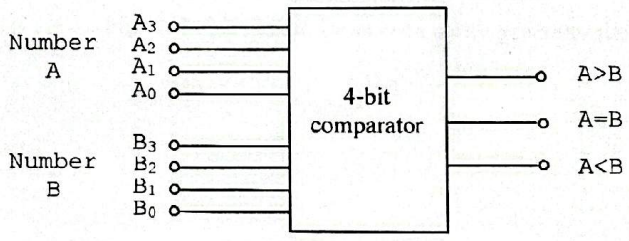


Fig.16(b)

Fig.16: (a) Logic circuit (b) Logic symbol

6.5 Parity Generator/Checker

In parity method, an extra bit known as parity bit is attached to a group of bits to make the total number of 1's in the group even or odd. Then the new group of bits (i.e. original group + parity bit) is transmitted to the receiver. The receiver then checks the evenness or oddness of the number of 1's in the new group received. Two different parity methods are used for error detection: i) Even parity method and ii) Odd parity method.

6.5.1 Even Parity Generator

In even parity method, the parity bit is made 1 if the group of bits being transmitted to the receiver contains an odd number of 1's. Otherwise, it is 0. In other words, the number of 1's in the group of bits along with the parity bit is made even. For example, suppose the group of bits 1000011 is to be transmitted to the receiver. This group contains three 1's. So, the parity bit, P_E will be 1 to make the number of 1's in the new group even i.e.

	P_E	1	0	0	0	0	1	1 ← Number of 1's is ODD
	↓							
New group of bits:	1	1	0	0	0	0	1	1 ← Number of 1's is made EVEN

Therefore, it needs a logic circuit that will produce a parity bit depending on the number of 1's in the original group of bits, so that, the total number of 1's in the group along with the parity bit is even. This logic circuit is called even parity generator.

Suppose we have to design an even parity generator for a 4-bit word (i.e. each group contains 4-bits of information). The block diagram for such a generator is shown in Fig.17 below.

CamScanner

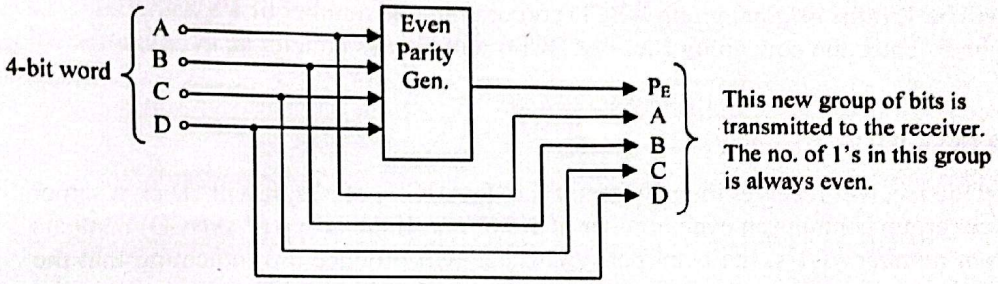
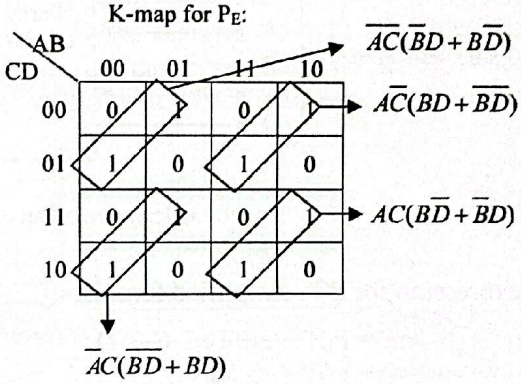


Fig.17: Block diagram of an even parity generator

Truth table of an even parity generator for a 4-bit word:

Inputs				Output
A	B	C	D	Even parity bit, P_E
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

K-map for P_E:



$$\therefore P_E = \overline{AC}(\overline{BD}+B\overline{D}) + \overline{AC}(BD+\overline{BD}) + AC(B\overline{D}+\overline{BD}) + \overline{AC}(\overline{BD}+BD)$$
$$= A \oplus B \oplus C \oplus D$$

The logic circuit is shown in Fig.18.

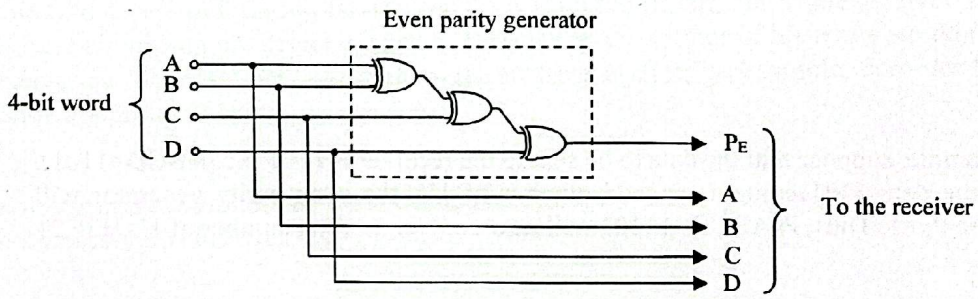


Fig.18: Logic circuit for an even parity generator for 4-bit word

P_E will be 1, if the original group ABCD contains an odd number of 1's. Otherwise, P_E will be 0. Thus, the new group (i.e. P_EABCD) will always contain an even number of 1's.

6.5.2 Even Parity Checker

When the receiver receives the new group (i.e. P_EABCD) of bits, it will check whether the new group contains an even number of 1's or not. If the group (P_EABCD) contains an even number of 1's, the even parity checker will produce a 0 indicating that the received data is correct. Otherwise, the checker will produce a 1 indicating that the received data is wrong. The logic symbol of an even parity checker is shown in Fig.19.

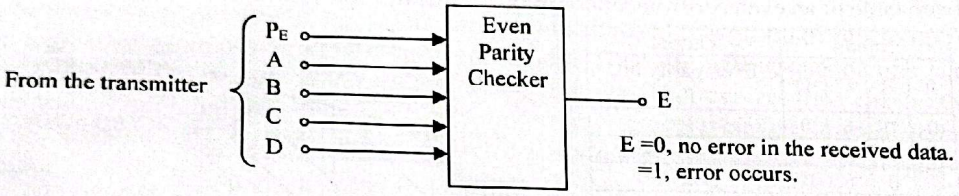


Fig.19: Logic symbol of an even parity checker

The expression for E in simplified form is,

$$E = P_E \oplus A \oplus B \oplus C \oplus D$$

The logic circuit is drawn below.

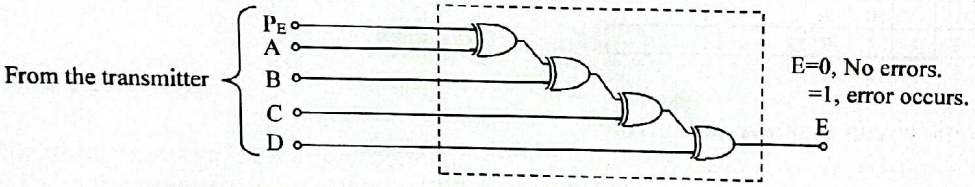


Fig.20: Logic circuit for an even parity checker

For example, suppose that the data to be sent to the receiver is 1101 (i.e. ABCD=1101). Since the data 1101 contains an odd number of 1's, the even parity generator will generate P_E=1. Thus, P_EABCD=11101 will now contain an even number of 1's [Fig.21 (a)].

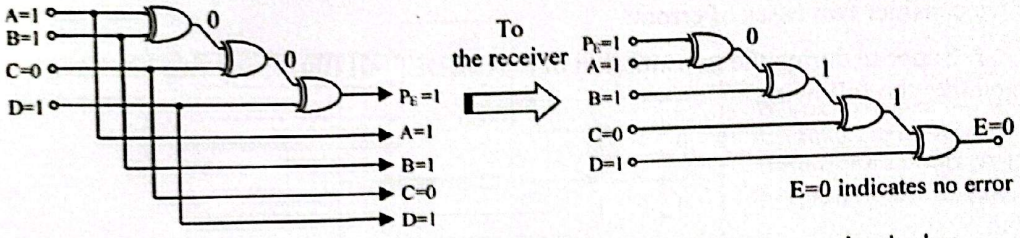


Fig. 21(a): Even parity generator

Fig. 21(b): Even parity checker

The receiver will receive P_EABCD=11101 from the transmitter. Then the even parity checker will check P_EABCD for errors. If no error has occurred, the output of the checker (E) will be 0 [Fig.21 (b)].

Suppose, for some noises during transmission, D has changed from 1 to 0. Thus, the receiver will receive P_EABCD=11100. This received data now contains an odd number of 1's. Therefore, the even parity checker will produce E=1 indicating error in the received data [Fig.22].

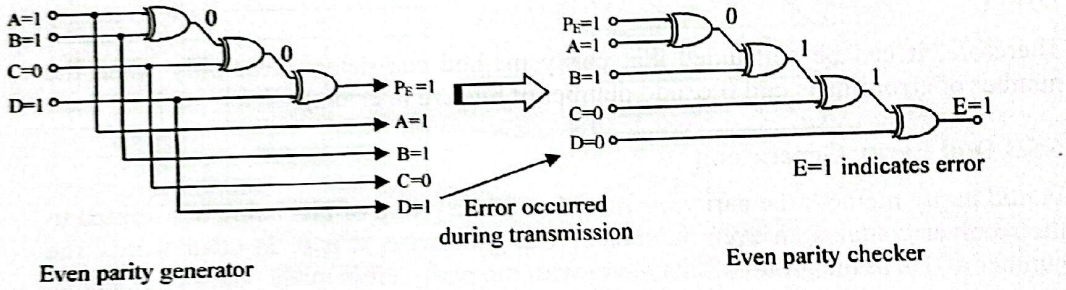


Fig.22

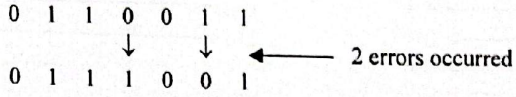
It can be mentioned here that if 2 bits, or 4 bits, or 6 bits i.e. even number of bits change from 0 to 1 or 1 to 0 during the transmission from the transmitter to the receiver, the checker circuit can not detect the error. Because, even number of bit errors would not change the 'oddness' or 'evenness' of the number of 1's. For example, consider the following group of bits:

ABCDEF=110011.

In this group the number of 1's is even. Therefore, P_E=0 and the new group to be sent to the receiver is P_EABCDEF=0110011.

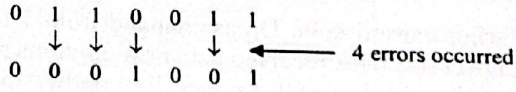
Here we consider two cases of errors:

I. Suppose during the transmission of $P_EABCDEF=0110011$, any two bits have changed like the following:



So, the new group if bits 0111001 will be received by the receiver and the checker circuit will produce E=0 indicating no errors. Here errors are not detected, because number of 1's in the received data 0111001 is still even. That is, errors in 2 bit positions can not change the oddness or evenness of 1's.

II. Suppose four bits have changed during transmission

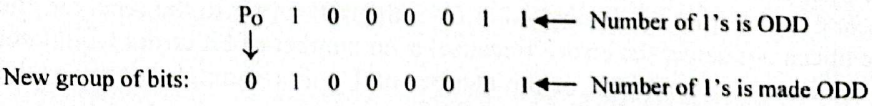


Here the error in the received data 0001001 can not also be detected by the checker. Because, the number of error bits is four (even). Thus, number of 1's in 0001001 is still even.

Therefore, it can be concluded that parity method can detect error only when the number of error bits is odd (i.e. odd number of bits are in errors).

6.5.3 Odd Parity Generator

In odd parity method, the parity bit is made 1 if the group of bits being transmitted to the receiver contains an even number of 1's. Otherwise, it is 0. In other words, the number of 1's in the group of bits along with the parity bit is made odd. For example, suppose the group of bits 1000011 is to be transmitted to the receiver. This group contains three 1's. So, the parity bit, P_O will be 0 to make the number of 1's in the new group odd i.e.



Thus, a logic circuit known as odd parity generator is needed to produce a parity bit depending on the number of 1's in the original group of bits, so that, the total number of 1's in the group along with the parity bit is made odd.

Suppose we have to design an odd parity generator for a 4-bit word (i.e. each group is of 4-bits). The block diagram for such a generator is shown below.

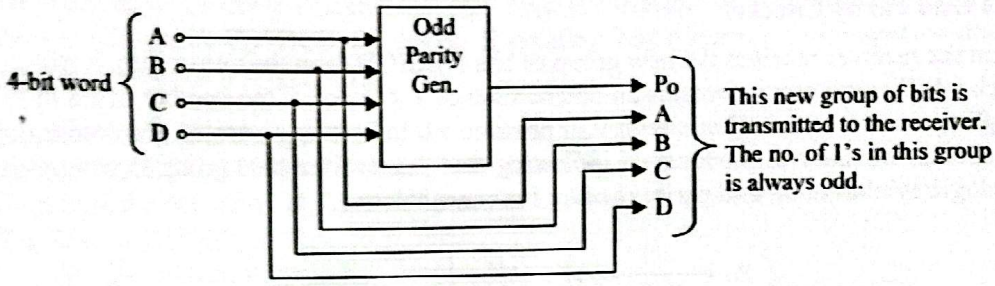
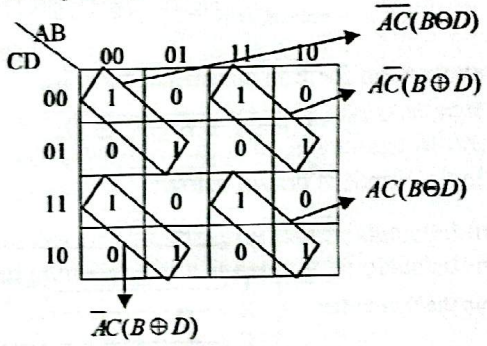


Fig.23: Block diagram of an odd parity generator

Truth table of an odd parity generator for a 4-bit word:

Inputs				Output
A	B	C	D	Odd parity bit, P_O
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

K-map for P_O:



$$\therefore P_O = \overline{AC}(B\Theta D) + A\overline{C}(B\oplus D) + AC(B\Theta D) + \overline{A}C(B\oplus D)$$
$$= (A\oplus C)\Theta(B\oplus D)$$

Logic circuit of odd parity generator:

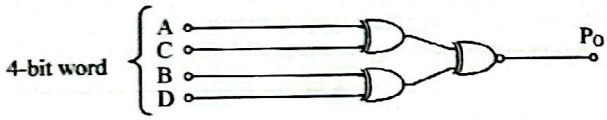


Fig.24: Logic circuit for an odd parity generator for 4-bit word

The odd parity generator will produce P_O=1, if the group of bits inputted (i.e. ABCD) contains an even number of 1's. Otherwise, P_O=0. Stated differently, the total number of 1's in the group along with the parity bit is made odd.

6.5.4 Odd Parity Checker

When the receiver receives the new group of bits P_OABCD from the transmitter, it will check whether the group contains an odd number of 1's or not. If the number of 1's in the group is odd, the checker circuit will produce a 0 indicating no errors. Otherwise, the checker circuit will produce a 1 indicating that the received data group is wrong. The logic symbol of an odd parity checker is shown below.

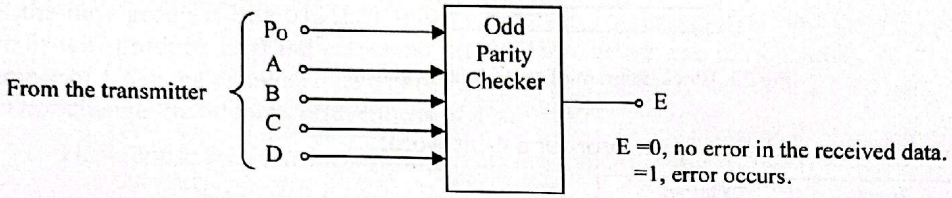


Fig.25: Logic symbol of an odd parity checker

The expression for E in simplified form is,

$$E = P_O \Theta (A \oplus B \oplus C \oplus D)$$

The logic circuit is drawn below:

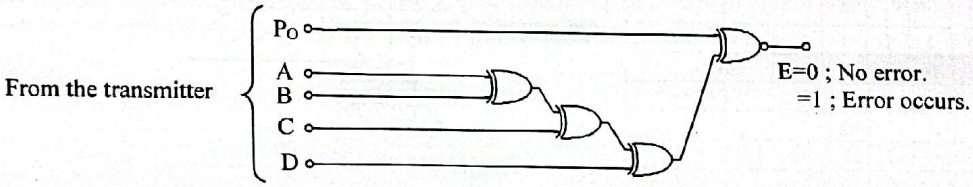
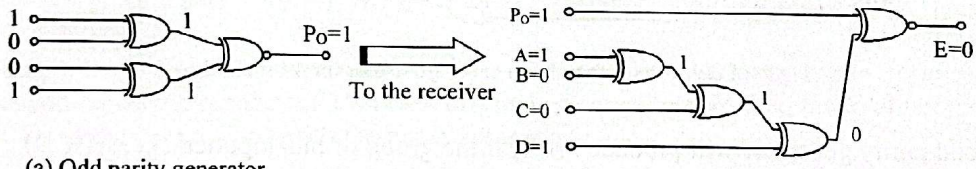


Fig.26: Logic circuit of an odd parity checker

For example, suppose that the group of data to be sent to the receiver is 1001 (i.e. ABCD=1001). Since the data 1001 contains an even number of 1's, the odd parity generator will generate P_O=1. Thus, P_OABCD=11001 will now contain an odd number of 1's [Fig.27 (a)].



(a) Odd parity generator

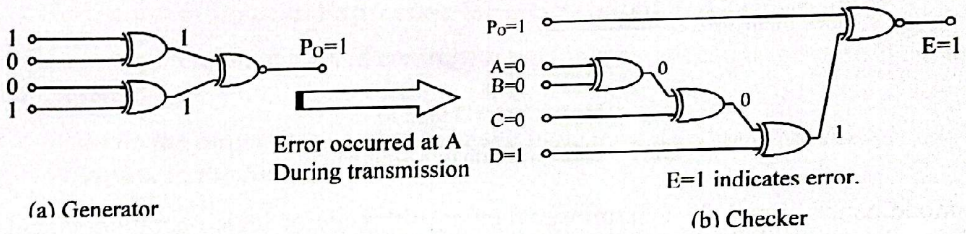
E=0 indicates no error

(b) Odd parity checker

Fig.27: Odd parity generator and checker

The receiver will receive $P_OABCD=11001$ from the transmitter. Then the odd parity checker will check P_OABCD for errors. If no error has occurred, the output of the checker (E) will be 0 [Fig.27 (b)].

Suppose that A has changed from 1 to 0 during transmission. Thus, the receiver will receive $P_OABCD=10001$. This received data has now an even number of 1's. Therefore, the odd parity checker will produce E=1 indicating error in the received data [Fig.28].



(a) Generator (b) Checker

Fig.28

When parity method is used, the transmitter and the receiver must have an agreement, in advance, to whether odd or even parity is being used. There is no advantage of one over the other.

If there is an error in one bit or in an odd number of bits, the error can be detected in parity method. But, if error occurs in an even number of bits, it can not be detected in this method.

6.6 Arithmetic Logic Unit

The Arithmetic Logic Unit (ALU) is a more complex logic circuit where both arithmetic operations (such as addition, subtraction, multiplication, etc) and logic operations (such as comparison, AND, OR, etc) are performed. In fact, this is a multi-function logic circuit. The various types of operations are usually selected by means of several control inputs. This is the fundamental building block of any microprocessor of a computer. Fig.29 shows the block diagram of a 4-bit arithmetic logic unit.

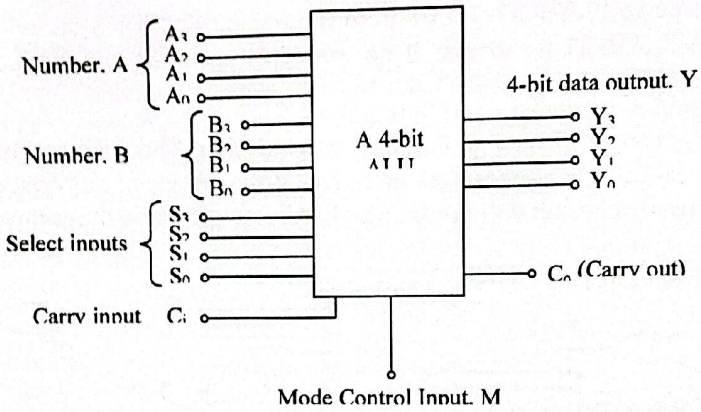


Fig.29: Block diagram of a 4-bit ALU

The mode control input, M determines whether arithmetic or logic operation should perform.

I) When M=1: The ALU receives two 4-bit numbers A and B from processor registers and performs logic operations. Between these two inputs A and B, there may be a total of 16 different types of logic outputs: 0, 1, A, B, A', B', A+B, $(A+B)'$, $A'+B$, $A+B'$, AB, $(AB)'$, $A'B$, AB', $A \oplus B$, and $(A \oplus B)'$. These outputs are selected by means of four select inputs (S_3, S_2, S_1, S_0) and are stored into ALU output registers. Other mechanisms move data between these registers and memory. To perform various logic operations, different types of logic circuits are built inside the ALU.

II) When M=0: The ALU receives data from processor registers and performs arithmetic operations. Depending on the value of the carry input C_i there may be a total of 32 arithmetic operations between two 4-bit numbers A and B: 16 operations when C_i=1 and 16 operations when C_i=0. These arithmetic operations are also selected by means of the select inputs (S_3, S_2, S_1, S_0). For every select input combination ($S_3S_2S_1S_0$), the arithmetic output when C_i=1 is always one greater than the corresponding output when C_i=0. Different types of digital circuits are built inside the ALU for performing arithmetic operations.

For processing word size larger than 4-bits, several 4-bit ALUs are to be cascaded by connecting the carry output C_0 of one ALU to the carry input C_i of the next ALU. The ALU is the heart of any microprocessor. In fact, a modern microprocessor like the Pentium may have multiple cores, each core with multiple execution units, and each unit with multiple ALUs. These ALUs are more powerful and complex logic circuits.

6.7 Analysis of Combinational Logic Circuits

The analysis of combinational logic circuits is somewhat the reverse procedure of the design procedure of combinational logic circuits. The design procedure starts from the specification of the problem and ends through the implementation of the logic circuit. On the other hand, the analysis procedure starts with a given logic circuit and ends with a set of Boolean expressions or a truth table.

6.7.1 Derivation of Boolean Expression for a Combinational Logic Circuit

The Boolean expression of a given combinational logic circuit can be derived using the following step:

"Write the output expression for each logic gate starting from the left-most inputs to the final output."

Example 6. Derive the output expression of the following logic circuit:

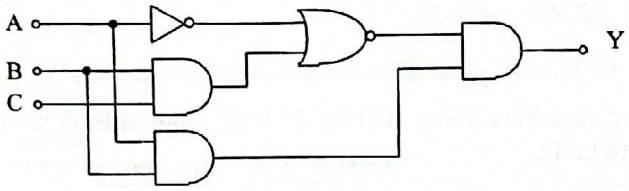


Fig.30

Solution:

i. Write the output expressions of the left-most gates (here one NOT and two AND gates). The expressions are $\overline{A}, BC, and\ AB$.

ii. Now write the output expression of the NOR gate. Here it is $\overline{\overline{A}+BC}$, and finally,

iii. Write the output expression of the AND gate: $\left(\overline{\overline{A}+BC}\right)AB$. This is the final output expression of the given logic circuit as shown in Fig.31.

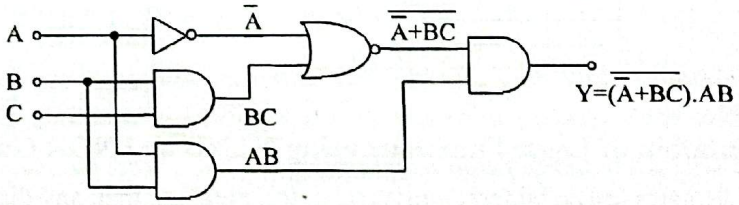


Fig.31

6.7.2 Derivation of the Truth Table for a Combinational Logic Circuit

Once the output expression of a combinational logic circuit is derived, the truth table can be constructed easily from the expression using the following steps:

i. List all possible combinations of values for the input variables in the input columns of the truth table and then
ii. Evaluate the expression for each of the input combinations and put the result in the output column of the truth table.

Example 7. Construct the truth table for the combinational logic circuit of Fig.30.
Solution: The output expression of the logic circuit of Fig.30 is:

$$\begin{aligned} Y &= \left(\overline{\overline{A} + BC}\right)AB \\ &= \left(A\overline{BC}\right)AB \\ &= A\left(\overline{B} + \overline{C}\right)AB \\ &= A\overline{B}.AB + A\overline{C}.AB \\ &= AB\overline{C} \end{aligned}$$

Because of 3 input variables, there will be 2^3 (=8) input combinations in the input columns of the truth table.

Now evaluate the output expression for each of 8 input combinations and put the result in the output column of the truth table. Here it is seen that only for input combination 110, the output is 1. For other combinations, the output is 0s. The final truth table is shown below.

Truth table

Input combination			Output,
A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

6.8 Implementation of Logic Functions using NAND and NOR Gates

NAND and NOR gates are considered universal gates, meaning that, any digital circuit can be implemented using only either NAND or NOR gates. Since these gates are easier to fabricate with electronic components, ICs are more frequently available in the

REDMI 10C

NAND and NOR forms rather than in the AND and OR forms. Therefore, it is important for us to be familiar with the techniques for implementing logic functions using NAND and NOR gates.

6.8.1 Two-level NAND Implementation

It is quite easy to implement any logic function using two-levels of NAND gates. The procedure for obtaining the NAND logic diagram from a logic function is as follows:

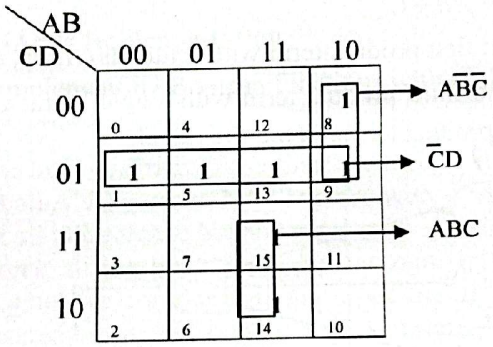
i. Simplify and express the function in SOP form.
ii. In the first level, draw a NAND gate for each product term of the function. The inputs to each of these NAND gates are the literals of the product term.
iii. In the second level, it requires only one NAND gate with the inputs coming from the outputs of the first level NAND gates.

Example 8. Implement the following function using only NAND gates

$$F(A,B,C,D)=\sum m(1,5,8,9,13,14,15)$$

Solution:

i. Simplify the function using K-map in SOP form as follows



Therefore, the function in simplified SOP form is

$$F(A,B,C,D) = ABC + A\overline{B}\overline{C} + \overline{C}D$$

ii. There are three product terms: first product term with 3 literals (A, B, C), second product term with 3 literals $(A,\overline{B},\overline{C})$, and third product term with 2 literals $(\overline{C},D)$. Therefore, we draw 3 NAND gates for 3 product terms in the first level.

15/07/2026

CamScanner

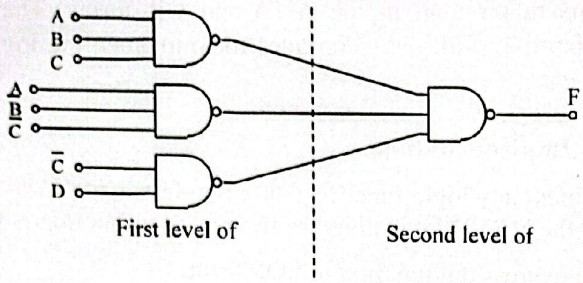


Fig.32

iii. In the second level, we draw only one NAND gate with the inputs coming from the outputs of the first level NAND gates. The final circuit is shown in Fig.32.

Example 9. Implement the following functions using only NAND gates

$i. F(A,B,C) = A\overline{B} + \overline{A}B + C$

$ii. F(A,B,C,D,E) = ABC + \overline{D} + \overline{E}$

Solution: $i. F(A,B,C) = A\overline{B} + \overline{A}B + C$

There are three product terms: first product term with 2 literals $(A, \overline{B})$, second product term with 2 literals $(\overline{A}, B)$, and third product term with single literal C. Therefore, it requires 3 NAND gates for 3 product terms in the first level.

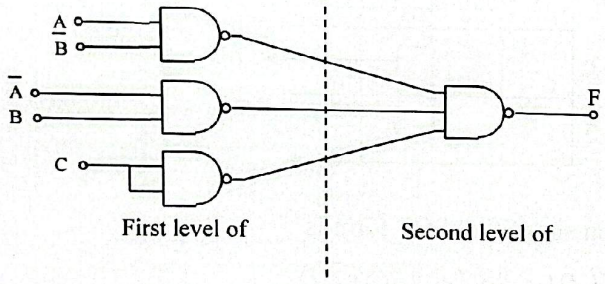


Fig.33

In the second level, draw only one NAND gate with the inputs coming from the outputs of the first level implementation (Fig.33).

ii. $F(A,B,C,D,E) = ABC + \overline{D} + \overline{E}$

There are three product terms: first product term with 3 literals (A, B, C), second product term with single literal $(\overline{D})$, and third product term with single literal $(\overline{E})$. Therefore, it requires 3 NAND gates for 3 product terms in the first level.

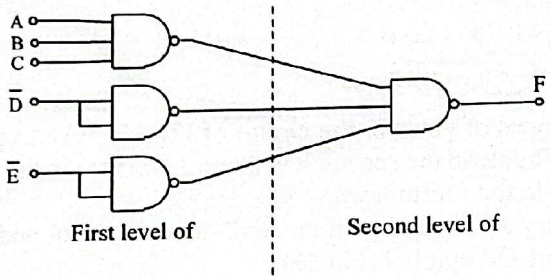


Fig.34

In the second level, draw only one NAND gate with the inputs coming from the outputs of the first level implementation (Fig.34).

6.8.2 Multi-level NAND Implementation

The procedure for implementing Boolean functions with multi-level NAND gates is as follows:

i. Draw the logic diagram of the function with AND and OR gates.
ii. Convert all AND gates to NAND gates with AND-invert symbols.
iii. Convert all OR gates to NAND gates with invert-OR symbols.
iv. Now check all the bubbles in the diagram. Insert an inverter (one-input NAND gate), or complement the input literal for the bubble that is not compensated by another bubble along the same line.

Example 10. Implement the following function using only NAND gates

$$F(A,B,C,D) = A(BC + \overline{B}\overline{C}) + \overline{C}D$$

Solution:

i. Draw the logic diagram of the function with AND and OR gates as shown in Fig.35 below:

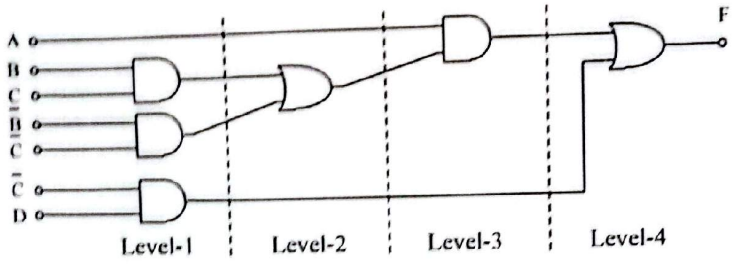


Fig.35

ii. There are four levels of gates in the circuit of Fig.35: 3 AND gates in the first level, one OR gate in the second level, one AND gate in the third level, and one OR gate in the fourth level.

iii. Now replace every AND gate with an AND-invert symbol and every OR gate with an invert-OR symbol (Fig.36).

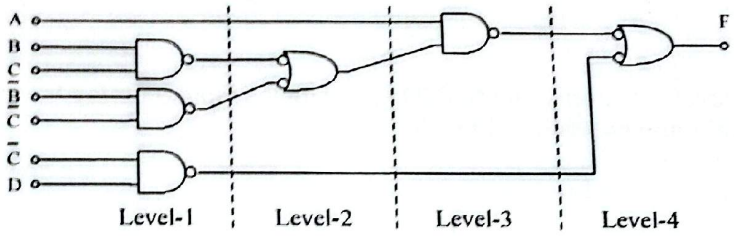


Fig.36

iv. Now check all the bubbles in Fig.36. Here every bubble is compensated by another one along the same line. Since the following two symbols in Fig.37 are equivalent according to DeMorgan's theorem, $\overline{A} + \overline{B} = \overline{AB}$, redraw the circuit of Fig.36 as shown in Fig.38.

Fig.37

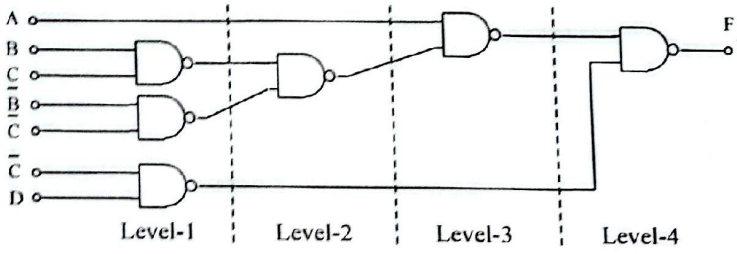


Fig 38: Implementation of the function F with multi-level NAND gates

Example 11. Implement the following function using only NAND gates

$$F(A,B,C,D) = (\overline{A}+C)(BC+D)$$

Solution:

i. Draw the logic diagram of the function with AND and OR gates as shown below:

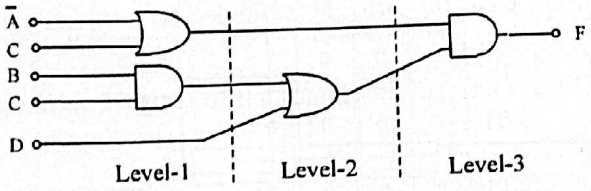


Fig.39

ii. There are three levels of gates in the circuit of fig.39: 2 gates in the first level, one gate in the second level, and one gate in the third level. Now replace every AND gate with an AND-invert symbol and every OR gate with an invert-OR symbol (Fig.40 (a)).

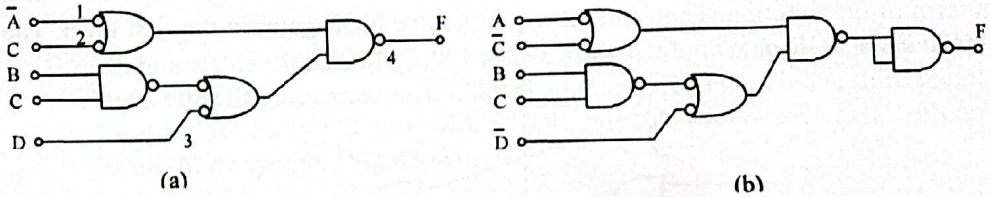


Fig.40

In Fig.40 (a), there are four lines (marked with labels 1, 2, 3, and 4) with single bubble. So, insert inverters (one-input NAND gates) along these lines or complement the literals. This is shown in Fig.40 (b).

6.8.3 Two-level NOR Implementation

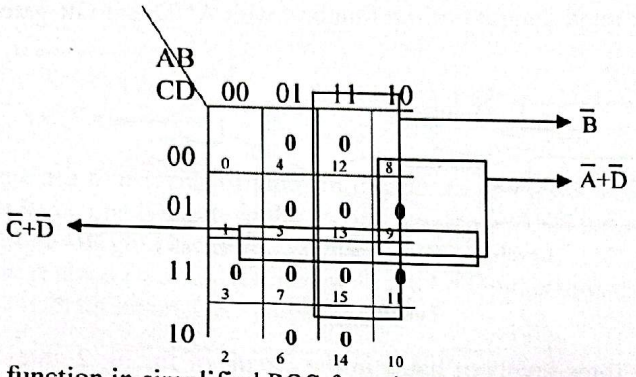
It is also quite easy to implement any logic function using two-levels of NOR gates if the function is in POS form. The procedure is as follows:

i. Simplify and express the function in POS form.
ii. In the first level, draw a NOR gate for each sum term of the function. The inputs to each of these NOR gates are the literals of the sum term.
iii. In the second level, it requires only one NOR gate with the inputs coming from the outputs of the first level NOR gates.

Example 12. Implement the following function using only NOR gates

$F(A, B, C, D)=\Pi M(3, 4, 5, 6, 7, 9, 11, 12, 13, 14, 15)$

Solution: Simplify the function using K-map in POS form:



Therefore, the function in simplified POS form is

$$F(A,B,C,D) = \overline{B}(\overline{A}+\overline{D})(\overline{C}+\overline{D})$$

There are three sum terms: $\overline{B}, (\overline{A}+\overline{D}), and\ (\overline{C}+\overline{D})$. Now draw a NOR gate for each sum term of the function. Therefore, it requires three NOR gates in the first level. The inputs to each NOR gate are the literals of the sum term.

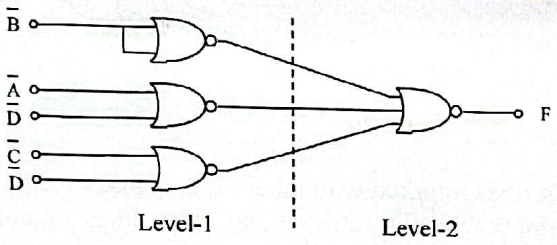


Fig.41

In the second level, draw only one NOR gate with the inputs coming from the outputs of the first level NOR gates. The final circuit is shown in Fig.41.

6.8.4 Multi-level NOR Implementation

The procedure for implementing Boolean functions with multi-level NOR gates is as follows:

i. Draw the logic diagram of the function with AND and OR gates.
ii. Convert all OR gates to NOR gates with OR-invert symbols.

iii. Convert all AND gates to NOR gates with invert-AND symbols.
iv. Now check all the bubbles in the diagram. Insert an inverter (one-input NOR gate) or complement the input literal for the bubble that is not compensated by another bubble along the same line.

Example 13. Implement the following function using only NOR gates

$$F(A,B,C,D) = (A\overline{B} + \overline{A}B)(C + \overline{D})$$

Solution:

i. Draw the logic diagram of the function with AND and OR gates as shown below:

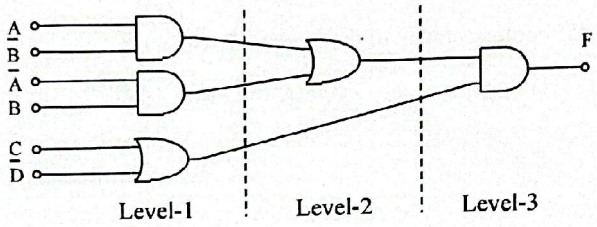


Fig.42

ii. There are three levels of gates in the circuit of Fig.42: 3 gates in the first level, one OR gate in the second level, and one AND gate in the third level. Now replace every OR gate with a NOR gate and every AND gate with an invert-AND symbol (Fig.43 (a)).

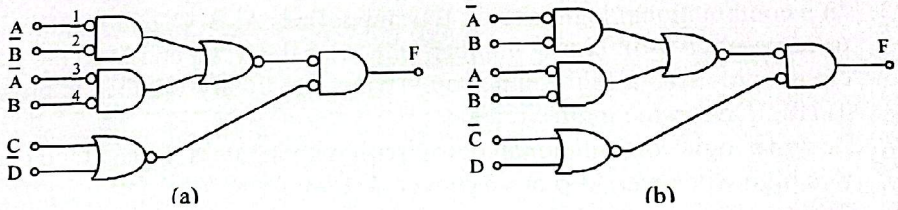


Fig.43

In Fig.43 (a) there are four lines (marked with 1, 2, 3, and 4) with single bubble. So, complement these literals as shown in Fig.43 (b).

Since the following two symbols in Fig.44 are equivalent according to DeMorgan's theorem, $\overline{AB} = \overline{A} + \overline{B}$, redraw the circuit of Fig.43 (b) as shown in Fig.45.

Fig.44

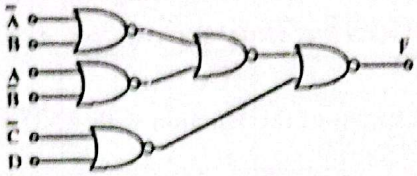


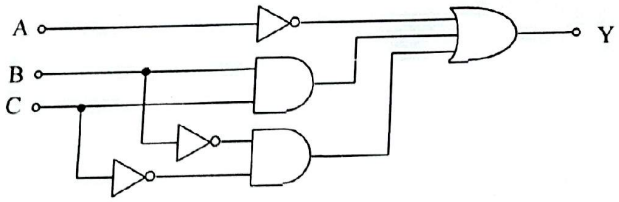
Fig.45: Implementation of the function F with multi-level NOR gates.

REVIEW QUESTIONS

1. What is a combinational logic circuit? List the steps involved in designing combinational logic circuits
2. Design a logic circuit that has four inputs (A, B, C, and D) and an output (Y), such that, the output is a 1 whenever the decimal equivalent of $ABCD)_2$ is divisible by either 3 or 5.
3. In a combinational logic circuit, four input lines A, B, C, and D are being used to represent a 4-bit binary number with A as the MSB and D as the LSB. The circuit produces a high output only when the binary number is greater than $0110)_2$. Design the logic circuit.
4. Design a logic combinational logic circuit with inputs A, B, and C so that output Y is high whenever A=0 or whenever B=C=1.
5. Design a logic circuit that accepts a 3-bit number and generates an output binary number equal to the square of the input number.
6. Design a logic circuit that accepts a 4-bit number and generates the 2's complement of the input number.
7. Design a logic circuit that will allow a signal A to pass to the output only when control inputs B and C are both high; otherwise, the output will stay low.
8. Design a logic circuit that has three inputs A, B, and C and whose output will be high only when a majority of the inputs is high.
9. What is a code converter?

10. Design a BCD-to-Binary code converter for two BCD code groups.
11. Design a binary-to-BCD code converter.
12. Design a 4-bit Binary-to-Gray code converter.
13. Design a 4-bit Gray-to-Binary code converter.
14. Design a BCD-to-Excess-3 code converter.
15. Design an Excess-3-to-BCD code converter.
16. Design a 3-bit/4-bit/5-bit magnitude comparator circuit.
17. Design a logic circuit for odd parity generator/checker for 4-bit message.
18. Design a 6-bit odd/even parity checker.
19. Draw and explain the block diagram of a 4-bit Arithmetic Logic Unit.
20. Realize each of the following functions using two-/multi-level NAND gates:
    i. $F(A,B,C,D) = A + B(\overline{C+D})$
    ii. $F(A,B,C,D) = A(BC + \overline{BC}) + \overline{C}D$
    iii. $F(A,B,C,D) = BD + BCD + A\overline{BC}\,\overline{D} + \overline{A}\overline{B}C\overline{D}.$
    iv. $F(A,B,C,D) = \sum m(1,5,8,9,13-15)$
    v. $F(A,B,C,D,E) = \overline{A} + BC + \overline{DE}$
    vi. $F(A,B,C,D,E) = A\overline{C} + ACE + AC\overline{E} + \overline{A}C\overline{D} + \overline{A}\overline{DE}$
21. Realize each of the following functions using two-/multi-level NOR gates:
    i. $F(A,B,C,D) = (A\overline{B} + \overline{A}B)(C\overline{D} + \overline{C}D)$
    ii. $F(A,B,C,D) = (A\overline{B} + \overline{A}B)(C + \overline{D})$
    iii. $F(A,B,C,D) = (\overline{B} + \overline{D})(\overline{A} + \overline{C} + D)(A + \overline{B} + \overline{C} + D)(\overline{A} + B + \overline{C} + \overline{D})$
    iv. $F(A,B,C,D) = \prod M(3-7,9,11-15)$

22. Derive the output expression of the logic circuit below.



23. Develop a truth table from the output expression of question no. 22.